10. Less, editors, and reading large output

10. Less, editors, and reading large output

Phase: Phase 3 — Text, pipes, and discovery
Estimated time: 1 day
Slug: `10-editors-paging`

Learning objectives

- Page output with less/more
- Edit files in nano or vim at a basic level
- Search inside less
- Avoid opening multi-GB logs in a GUI editor

Prerequisites

Complete prior numbered lessons. Prefer a real Linux VM/container for Linux-specific topics. Type every command yourself — muscle memory beats skimming.

Why this matters

On a remote server you often have no graphical editor. You must read long logs, tweak a config and save it using only a terminal. Knowing how to page through output with `less` and how to survive in `vim` (including how to quit without saving) is a basic survival skill; most people first meet vim by accident and get stuck.

Core concepts

`less` is a pager: it shows a file one screen at a time, does not load everything into memory, and lets you search backwards and forwards. It is the default pager for `man` and `git log`.

less key	Action
Space / `f`	Next page
`b`	Previous page
`g` / `G`	Top / bottom
`/text` then `n` / `N`	Search forward, next / previous match
`?text`	Search backward
`F`	Follow like `tail -f` (Ctrl-C to stop)
`-N`	Toggle line numbers
`q`	Quit

Editors: `nano` is simple (commands shown at the bottom, `^O` write, `^X` exit). `vi`/`vim` is on nearly every system and is *modal*.

```
           i, a, o                    :  (colon)
NORMAL  ------------>  INSERT      NORMAL ------> COMMAND-LINE
(move, delete,   <----  (type text)              (:w  :q  :wq  :q!)
 copy, paste)    Esc
```

Normal mode is the default; keys are commands (`dd` delete line, `yy` copy line, `p` paste, `u` undo, `x` delete char, `/word` search). `i` enters insert mode; `Esc` returns to normal mode.

Command-line	Meaning
`:w`	Write (save)
`:q`	Quit (refuses if unsaved changes)
`:wq` or `ZZ`	Save and quit
`:q!`	Quit and discard changes
`:w !sudo tee %`	Save a root-owned file you opened without sudo
`:set number`	Show line numbers

The escape route when stuck: press `Esc` a couple of times, then type `:q!` and Enter to leave without saving.

Worked examples

Page through a big file:

```
less /var/log/syslog
# sample output: Oct 21 10:00:01 box CRON[1234]: (root) CMD (...)
# sample output: (END)
```

Inside less, type `/error` Enter to search and `n` for the next hit, `q` to exit.

Pipe into the pager:

```
dmesg | less -S
ls -l /usr/bin | less
```

`-S` chops long lines instead of wrapping them.

Editing with vim, a typical sequence:

```
vim notes.txt
# sample output: (vim opens; press i, type "hello", press Esc)
# sample output: :wq   (then Enter) saves and exits
```

Quit without saving:

```
vim notes.txt
# sample output: (you changed text by mistake)
# sample output: Esc, then :q!  and Enter  -> discards changes
```

Choosing your default editor and pager:

```
export EDITOR=nano
export PAGER=less
echo "$EDITOR"
# sample output: nano
```

Programs such as `git commit` and `crontab -e` honor `$EDITOR` (or `$VISUAL`).

Nano quick use:

```
nano notes.txt
# sample output: (type text, then Ctrl-O Enter to save, Ctrl-X to exit)
```

Mini-lab

1. `cd /tmp && seq 1 500 > big.txt && less big.txt`; press `G`, `g`, then `/250` Enter, then `q`.
2. Run `less +F /var/log/dpkg.log` (or any log), see it follow, press Ctrl-C then `q`.
3. Run `vim hello.txt`, press `i`, type two lines, press `Esc`, type `:wq` Enter.
4. `cat hello.txt` to verify the content.
5. Reopen with `vim hello.txt`, press `dd` to delete a line, then `:q` (it refuses) and `:q!` to discard.
6. Reopen, press `yy` then `p` to duplicate a line, `u` to undo, `:x` to save and exit.
7. In vim run `:set number`, search with `/two`, go to line 1 with `gg`.
8. Set `export EDITOR=nano` and run `crontab -e` or `visudo` mentally noting which editor opens; quit without changes.

Decision guide / failure modes

Use `less` for reading, `nano` when you need a quick edit and want on-screen hints, `vim` when it is the only editor available or you want speed with practice. Use `sed -i` for scripted edits rather than opening an editor.

Common failures:

- Stuck in vim: you are in insert mode or the command line; press `Esc` and use `:q!` to quit without saving or `:wq` to save.
- Typing text in normal mode and triggering random commands; check for `-- INSERT --` at the bottom.
- “E37: No write since last change” means use `:q!` or save first.
- Editing a root-owned file without sudo: you get “E45: readonly”; use `sudoedit` or `:w !sudo tee %`.
- Swap file warnings after a crash: another session may still have the file open; check before deleting `.swp`.
- Using `cat` on huge files and flooding the terminal; use `less` or `head`.

Common mistakes

- cat-ing multi-GB files to the terminal
- Getting stuck in vim without knowing `:q!`

- Editing production configs without backups

Exercises

Work in `exercises/10-editors-paging/`. Read that folder's `README.md` fully before coding.

```
cd exercises/10-editors-paging
chmod +x test.sh exercise.sh
./test.sh
```

What you will build

Implement vim_quit_nosave(). It takes no arguments and returns the exact string typed in vim's command-line mode to quit without saving changes, which is ":q!" (colon, letter q, exclamation mark). Return it as a plain string with no newline, no spaces, and no Esc character. The exercise is a memory anchor: when you are stuck in vim after accidental edits, press Esc, then type this and press Enter. Remember that ":wq" saves and quits, ":q" refuses with unsaved changes, and the "!" forces the quit.

Study loop

- **A (Drill):** make `./test.sh` pass
- **B (Apply):** run 2–3 lab commands from this lesson on a real Linux host; paste notable output into your notes
- **C (Challenge):** rewrite the exercise from memory the next day

Do not open `solutions/10-editors-paging/` until you have an honest failing attempt.

Self-check

Try to answer without scrolling up:

1. How search in less?
2. How quit vim without saving?
3. Why page instead of cat?

Further reading

- https://www.vim.org/